

CapyToons.to

A Comic Reading Platform

Database Systems Lab — Project Proposal (BSSE-B)

Student 1: Hadia Khan

Student 2: Hamna Rehman

Introduction & Background

The global manga, manhwa, manhwa, and webtoon industry has expanded into a multi-billion-dollar market, with millions of readers worldwide consuming digital comics daily. Platforms that aggregate content from diverse origins (Japan, Korea, China) have become essential hubs for fan communities. Despite this growth, many popular platforms such as *bato.to* have experienced prolonged outages or shutdowns, leaving large reader bases without a reliable alternative. A well-designed, database-driven reading platform is essential to serve this audience consistently.

Problem Statement

- Major free comic platforms (e.g., *bato.to*) frequently go offline, leaving millions of readers stranded with no fallback.
- Existing alternatives offer poor filtering, making it hard to discover titles by genre, language, status, or length.
- Reading history, bookmarks, and preferences are rarely preserved across sessions for registered users.
- Without a robust database backend, metadata inconsistency (wrong chapter counts, missing genres) is common.
- **Who is affected:** Casual readers, hobbyists, fan-translators, and content curators globally.

Proposed Solution

CapyToons.to is a web-based comic reading platform backed by a relational database. It aggregates manga, manhwa, manhwa, and webtoons under a single interface. Registered users receive personalized recommendations based on reading history and ratings. A powerful multi-filter search engine allows discovery by genre, type, status, language, year, and author. The system prioritises data integrity, fast retrieval, and a smooth reading experience.

Objectives of the System

- Provide a centralised, reliable repository for comic metadata and chapters.
- Enable personalised content recommendations for registered users.
- Support advanced multi-attribute search and filtering.
- Ensure data consistency across comic entries, chapters, and user interactions.
- Facilitate fast CRUD operations for content management.
- Generate trending and popularity reports based on view counts and ratings.

Scope of the Project

Included:

- User registration, login, and profile management.
- Comic catalogue with full metadata storage.
- Chapter-level reading with page navigation.
- Advanced search with filters (type, genre, status, year, language, chapter count, author/title).
- Personalised home page: trending, most-viewed, and suggested comics.
- Ratings and reviews system.
- Bookmark / reading list functionality.

Excluded:

- Mobile native applications (web only).
- Content uploading by general public (admin-managed).
- Payment or premium subscription tiers (initial release).

Role of the Database System

The database is the core of CapyToons. It stores: comic metadata (title, author, release year, genres, status, description, cover image), chapter records, user accounts, reading progress, bookmarks, ratings, and reviews. A relational database enforces referential integrity between users, comics, and chapters. CRUD operations support content management; aggregate queries power trending and personalised feeds. Constraints (NOT NULL, UNIQUE, FK) ensure data quality.

System Features / Functional Requirements

1. **User Authentication** — Register, login, logout, and profile editing.
2. **Comic Detail Page** — Displays name, cover, author, release date, genres, status, chapter count, rating, description, and reviews.
3. **Chapter Reader** — Page-by-page or vertical scroll reading with progress tracking.
4. **Advanced Search & Filter** — Filter by type (manga/manhwa/manhua/webtoon), genre, status (ongoing, completed, hiatus, cancelled, stopped updating), year, language, chapter count range, and search by title or author name.
5. **Home Page Feed** — Trending comics, most-viewed comics, and personalised suggestions for logged-in users.
6. **Ratings & Reviews** — Users can rate comics (1–5 stars) and write text reviews.
7. **Bookmarks / Reading Lists** — Save comics to custom lists (e.g., Reading, Plan to Read, Completed, Dropped).
8. **Reading History** — Automatically tracks last-read chapter per comic per user.
9. **Admin Content Management** — Add, edit, or remove comics and chapters via admin panel.
10. **Reporting** — View-count analytics and top-rated / top-trending reports.

Preliminary Data Design

- **User** (UserID, Username, Email, PasswordHash, JoinDate, AvatarURL)
- **Comic** (ComicID, Title, CoverImageURL, Description, ReleaseYear, Status, Type, Language, AuthorID)
- **Author** (AuthorID, Name, Nationality)
- **Genre** (GenreID, GenreName)
- **ComicGenre** (ComicID, GenreID) — *many-to-many*
- **Chapter** (ChapterID, ComicID, ChapterNumber, Title, UploadDate, PageCount)
- **Review** (ReviewID, UserID, ComicID, Rating, ReviewText, ReviewDate)
- **Bookmark** (UserID, ComicID, ListType)

- **ReadingHistory** (UserID, ComicID, LastChapterID, LastReadAt)
- **ComicViews** (ComicID, TotalViews, WeeklyViews)

Key relationships: *Comic* → *Author* (*many-to-one*); *Comic* ↔ *Genre* (*many-to-many*); *User* → *Review*, *Bookmark*, *ReadingHistory* (*one-to-many*); *Comic* → *Chapter* (*one-to-many*).

Existing Systems & Comparative Analysis

Platform	Limitation
bato.to	Currently offline; under a lawsuit.
MangaFire.to	Good UX; poor reviews management.
MangaDex	Annoying Ads/Pop-ups; bad availability of content.
Webtoon (official)	Only hosts select licensed titles; no community uploads.

CapyToons addresses these gaps with reliable hosting, advanced filtering, personalised recommendations, and a unified catalogue across all comic types.

Proposed Technology Stack

Frontend: React.js, Tailwind CSS
Backend: Node.js with Express.js
Database: MySQL (primary relational DB)
Tools: MySQL Workbench, Postman, Git/GitHub, Figma (UI mockups), VS Code

Expected Outcomes

CapyToons.to will be a functional prototype of a comic reading platform demonstrating core database design principles. The system will support user registration, comic browsing, chapter reading, and basic search and filtering. The database will reflect a well-normalised relational schema with proper use of primary keys, foreign keys, and constraints to ensure data integrity. CRUD operations will be implemented for content and user management. The project will serve as a practical demonstration of how a relational database can be designed and integrated into a real-world inspired web application.